

Exam Revision 1 — Formal Algorithm Analysis

VCE Algorithmics (HESS) · Unit 4 AOS1 drills · answers at the back

A — Rapid-fire bounds (tightest Big-O, one-line justification each)

A1. $O(7n^2 + 3n \log n + 90)$

A2. $O\left(\frac{n(n-1)}{2} + n \log n\right)$

A3. $O(2^n + n^{1000})$

A4. $O(n \cdot m + n^2)$, m independent of n

A5. Loop with $i \leftarrow i * 5$ to reach n

A6. Two sequential loops of n then n^2

A7. n iterations, each doing a binary search on n items

A8. BFS on a complete graph, in $|V|$ only

B — Pseudocode analysis

B1.

```
1 Algorithm Filter(L):
2   kept ← empty list
3   Foreach x in L Do
4     If good(x) Do           # good() is O(log n)
5       append x to kept
6   Return kept
```

B2.

```
1 Algorithm Pairs(L):
2   Foreach i from 1 To n Do
3     Foreach j from i+1 To n Do
4       If L[i] + L[j] = 0 Do
5         Return (i, j)
6   Return none
```

B3.

```
1 Algorithm Chunks(L):
2   size ← n
3   While size > 1 Do
4     Foreach x in L Do
5       print(x, size)
6   size ← size div 2
```

C — Recurrences and Master Theorem

C1. $T(n) = 2T(n/2) + n$

C2. $T(n) = 9T(n/3) + n$

C3. $T(n) = T(n/2) + n$

C4. $T(n) = T(n-1) + 1$

C5. $T(n) = 2T(n-1) + 1$

C6. $T(n) = 3T(n/4) + n^2$

D — Definitions and cases

D1. Show $f(n) = 8n + 40$ is $O(n)$ from the formal definition (state c , n_0 , verify).

D2. Early-exit linear search: best case, worst case, and why the headline bound is the worst case.

D3. Why is “ $O(n^3)$ ” a technically-true but poor answer for an algorithm that is $\Theta(n \log n)$?

D4. Dijkstra (heap) vs Dijkstra (array): when is the array version preferable, and why?

Answers

A1 $O(n^2)$ — dominant term. **A2** $O(n^2)$ — triangular sum expands to $n^2/2$ term. **A3** $O(2^n)$ — exponential beats any polynomial. **A4** $O(nm + n^2)$ — two independent sizes; neither term drops without a relation between m and n . **A5** $O(\log n)$ — multiplying by a constant reaches n in $\log_5 n$ steps. **A6** $O(n^2)$ — sequence adds, dominant term kept. **A7** $O(n \log n)$ — n iterations $\times \log n$ each. **A8** $O(|V|^2)$ — $|E| = \Theta(|V|^2)$ on a complete graph.

B1 $O(n \log n)$ — n iterations, each dominated by the $O(\log n)$ call; append is $O(1)$. **B2** worst case $O(n^2)$ — triangular double loop, early Return only helps the best case ($\Theta(1)$). **B3** $O(n \log n)$ — outer loop halves size: $\log n + 1$ passes, each $O(n)$.

C1 $a=2, b=2$: $n^{\log_2 2} = n = f$ — case 2 — $\Theta(n \log n)$. **C2** $n^{\log_3 9} = n^2$ vs n — case 1 — $\Theta(n^2)$. **C3** $n^{\log_2 1} = 1$ vs n — case 3 — $\Theta(n)$. **C4** form fails; expansion: $\Theta(n)$. **C5** form fails; expansion: $\Theta(2^n)$. **C6** $n^{\log_4 3} \approx n^{0.79}$ vs n^2 — case 3 — $\Theta(n^2)$.

D1 e.g. $c = 48$, $n_0 = 1$: for $n \geq 1$, $40 \leq 40n$, so $8n + 40 \leq 48n$. **D2** Best $\Theta(1)$ (first element matches); worst $\Theta(n)$ (absent); the worst case is a guarantee over all inputs, so it is the quotable bound. **D3** Loose bounds convey less information; exam marking rewards the tightest provable bound. **D4** Dense graphs: array's $O(|V|^2)$ beats heap's $O((|V| + |E|) \log |V|) = O(|V|^2 \log |V|)$ when $|E| = \Theta(|V|^2)$.